

HARDA – Hazard Assessment Road Detection Application

ALI HARIZ BIN ANUARI

UNIVERSITI TEKNOLOGI MALAYSIA

DECLARATION OF THESIS / UNDERGRADUATE PROJECT REPORT AND COPYRIGHT

Author's full name : ALI HARIZ BIN ANUARI

Date of Birth : 31/08/2003

Title : HARDA – Hazard Assessment Road Detection Application

Academic Session :

I declare that this thesis is classified as:

☐

CONFIDENTIAL (Contains confidential information under the Official Secret Act 1972)*

☐

RESTRICTED (Contains restricted information as specified by the organization where research was done)*

☒

OPEN ACCESS I agree that my thesis to be published as online open access (full text)

1. I acknowledged that Universiti Teknologi Malaysia reserves the right as follows:
- The thesis is the property of Universiti Teknologi Malaysia
 - The Library of Universiti Teknologi Malaysia has the right to make copies for the purpose of research only.
 - The Library has the right to make copies of the thesis for academic exchange.

Certified by:



SIGNATURE OF STUDENT

030831140221

MATRIX NUMBER

A22EC0037

Date: 26 JUNE 2025



SIGNATURE OF SUPERVISOR

DR. MUHAMMAD
KHATIBSYARBINI BIN ABD
RAHIM

NAME OF SUPERVISOR

Date: 26 JUNE 2025

NOTES : If the thesis is CONFIDENTIAL or RESTRICTED, please attach with the letter from the organization with period and reasons for confidentiality or restriction

“I hereby declare that we have read this thesis and in my
opinion this thesis is sufficient in term of scope and quality for the
award of the degree of Bachelor of Computer Science (Computer Networks &
Security)”

Signature

:



Name of Supervisor

: MUHAMMAD KHATIBSYARBINI BIN ABD
RAHIM

Date

: 26 JUNE 2025

HARDA – Hazard Assessment Road Detection Application

ALI HARIZ BIN ANUARI

A thesis submitted in fulfilment of the
requirements for the award of the degree of
Bachelor of Computer Science (Software Engineering)

Faculty of Computing
Universiti Teknologi Malaysia

JUNE 2026

DECLARATION

I declare that this thesis entitled “*HARDA – Hazard Assessment Road Detection Application*” is the result of my own research except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature :

A handwritten signature in black ink, appearing to read 'Ali Hariz', with a stylized star-like symbol to the left and a horizontal line underneath.

Name : ALI HARIZ BIN ANUARI

Date : 26 JUNE 2025

DEDICATION

I dedicate this thesis on HARDA – Hazard Assessment Road Detection Application to my beloved parents, whose unwavering love, patience and encouragement have been my greatest source of strength throughout this journey. Their sacrifices and support have made this achievement possible.

ACKNOWLEDGEMENT

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Muhammad Khatibsyarbini Bin Abd Rahim, for his invaluable guidance, insightful feedback, and continuous support throughout the development of HARDA – Hazard Assessment Road Detection Application. His expertise, patience, and encouragement have been instrumental in shaping this work and helping me overcome various challenges during my research journey.

I am also profoundly thankful to my parents, whose unwavering love, understanding, and belief in me have been a constant source of motivation. Their sacrifices and encouragement have given me the strength to persevere and strive for excellence in all my academic endeavours.

Furthermore, I would like to sincerely acknowledge my peers and colleagues in the Computer Science department for their friendship, constructive discussions, and collaborative spirit. Their willingness to share knowledge, provide feedback, and offer moral support has greatly enriched my learning experience and made this research project more fulfilling. The stimulating academic environment and peer encouragement played a crucial role in maintaining my enthusiasm and commitment throughout this journey.

ABSTRACT

Road safety in Malaysia is severely affected by prevalent road hazards such as potholes, faded or blurry lane markings, and uneven surfaces, which substantially contribute to traffic accidents, vehicle damage, and overall degradation of transportation infrastructure. Traditional hazard reporting systems rely extensively on manual user input, which often leads to inaccurate, inconsistent, and delayed data, hampering timely intervention by authorities. This research introduces HARDA (Hazard Assessment Road Detection Application), an innovative system that integrates advanced computer vision and deep learning techniques to automate the detection and classification of road hazards from images submitted by users via mobile and web platforms.

At the core of HARDA is the YOLO (You Only Look Once) algorithm, a state-of-the-art real-time object detection model known for its superior accuracy and high inference speed. YOLO's architecture enables the system to simultaneously detect multiple hazards in a single pass, processing images efficiently to identify potholes, lane markings, and uneven surfaces with high precision. This approach significantly reduces the dependency on manual input, minimizing human error and accelerating the reporting process. The detected hazards are geotagged using GPS metadata and visualized on an interactive map through integration with the Google Maps API, allowing users and local authorities to monitor hazard locations dynamically.

The project encompasses the design and development of a robust system architecture supporting mobile and web interfaces, real-time image processing pipelines, and an administrative dashboard for validation and prioritization of reports. Performance evaluation includes quantitative assessment of YOLO's detection accuracy, system latency, and usability testing to ensure the application meets operational requirements. Preliminary results indicate that the HARDA system achieves high detection accuracy with low latency, demonstrating its capability to provide reliable, timely, and actionable hazard data.

By leveraging YOLO's real-time detection capabilities and combining it with geospatial mapping, HARDA contributes to enhancing road safety management and aligns with Malaysia's digital transformation initiatives aimed at building smart cities. The system offers a scalable and efficient solution for proactive infrastructure maintenance, ultimately aiming to reduce accidents and improve the overall quality of road networks.

ABSTRAK

Keselamatan jalan raya di Malaysia terjejas teruk akibat kewujudan *road hazard* seperti lubang jalan, garisan lorong yang pudar atau kabur, serta permukaan jalan yang tidak rata, yang banyak menyumbang kepada kemalangan lalu lintas, kerosakan kenderaan, dan kemerosotan infrastruktur pengangkutan secara keseluruhan. Sistem pelaporan *road hazard* tradisional bergantung sepenuhnya kepada input manual pengguna, yang sering menghasilkan data yang tidak tepat, tidak konsisten, dan lewat, sekaligus menghalang tindakan segera oleh pihak berkuasa. Projek ini memperkenalkan HARDA (*Hazard Assessment Road Detection Application*), sebuah sistem inovatif yang mengintegrasikan teknik penglihatan komputer dan pembelajaran mendalam terkini untuk mengautomasi pengesanan dan pengkelasan bahaya jalan daripada imej yang dihantar oleh pengguna melalui platform mudah alih dan web.

Teras utama HARDA adalah algoritma YOLO (*You Only Look Once*), model pengesanan objek masa nyata yang canggih yang terkenal dengan ketepatan tinggi dan kelajuan inferens yang pantas. Seni bina YOLO membolehkan sistem mengesan pelbagai bahaya dalam satu proses, memproses imej secara cekap untuk mengenal pasti lubang jalan, garisan lorong, dan permukaan tidak rata dengan ketepatan yang tinggi. Pendekatan ini secara signifikan mengurangkan kebergantungan pada input manual, meminimalkan kesilapan manusia dan mempercepatkan proses pelaporan. Bahaya yang dikesan akan ditandakan dengan koordinat GPS dan divisualisasikan pada peta interaktif melalui integrasi dengan API Google Maps, membolehkan pengguna dan pihak berkuasa tempatan memantau lokasi bahaya secara dinamik.

Projek ini merangkumi reka bentuk dan pembangunan seni bina sistem yang mantap menyokong antara muka mudah alih dan web, saluran pemprosesan imej masa nyata, serta papan muka pentadbiran untuk pengesanan dan keutamaan laporan. Penilaian prestasi meliputi penilaian kuantitatif ketepatan pengesanan YOLO, latensi sistem, dan ujian kebolegunaan bagi memastikan aplikasi memenuhi keperluan operasi. Keputusan awal menunjukkan sistem HARDA mencapai ketepatan pengesanan yang tinggi dengan latensi rendah, menunjukkan kebolehannya menyediakan data bahaya yang boleh dipercayai, tepat pada masa, dan boleh diambil tindakan.

Dengan memanfaatkan kebolehan pengesanan YOLO dan menggabungkannya dengan pemetaan geospasial, HARDA menyumbang kepada peningkatan pengurusan keselamatan jalan raya dan sejajar dengan inisiatif transformasi digital Malaysia yang bertujuan membangunkan bandar pintar. Sistem ini menawarkan penyelesaian yang boleh diskala dan cekap untuk penyelenggaraan infrastruktur proaktif, bertujuan mengurangkan kemalangan dan meningkatkan kualiti rangkaian jalan secara keseluruhan.

TABLE OF CONTENTS

	TITLE	PAGE
	DECLARATION	ii
	DEDICATION	iii
	ACKNOWLEDGEMENT	iv
	ABSTRACT	v
	ABSTRAK	vi
	TABLE OF CONTENTS	vii
	LIST OF TABLES	x
	LIST OF FIGURES	xi
	LIST OF ABBREVIATIONS	xii
	LIST OF APPENDICES	xiii
CHAPTER 1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Background	1
1.3	Project Aim	2
1.4	Project Objectives	2
1.5	Project Scope	3
1.6	Project Importance	3
1.7	Report Organization	3
CHAPTER 2	LITERATURE REVIEW	5
2.1	Introduction	5
2.2	Current System Analysis	5
2.3	Comparison between existing systems	10
2.4	Literature Review of Technology Used	11
2.4.1	YOLO (You Only Look Once) Object Detection Algorithm	11
2.4.2	Google Maps API for Geospatial Mapping	12

2.4.3	Mobile and Web Development Frameworks	12
2.4.4	Image Preprocessing and Data Handling	12
2.5	Chapter Summary	13
CHAPTER 3	SYSTEM DEVELOPMENT METHODOLOGY	15
3.1	Introduction	15
3.2	Methodology Choice and Justification	15
3.3	Phases of Chosen Methodology	16
3.3.1	Requirement Analysis Phase	16
3.3.2	Design Phase	16
3.3.3	Implementation Phase	17
3.3.4	Testing Phase	17
3.3.5	Deployment and Maintainance	18
3.4	Technology Used Description	19
3.4.1	System Design Tool	19
3.4.2	Project Management Tool	19
3.4.3	Implementation Environment	19
3.5	System Requirement Analysis	20
3.5.1	Functional Requirements	20
3.5.2	Non-Functional Requirements	21
3.5.3	Hardware Requirements	21
3.5.4	Software Requirements	22
3.6	Chapter Summary	23
CHAPTER 4	REQUIREMENT ANALYSIS AND DESIGN	24
4.1	Introduction	24
4.2	Requirement Analysis	24
4.2.1	Functional Requirement	24
4.2.2	Non-Functional Requirement	27
4.3	Project Design	27
4.4	Database Design	29
4.5	Interface Design	30
4.5.1	Mobile Application Interface Design	30

4.5.2	Web Application Interface Design	30
4.6	Chapter Summary	31
CHAPTER 5	IMPLEMENTATION AND TESTING	33
CHAPTER 6	CONCLUSION	33
6.1	Introduction	33
6.2	Achievements of Project Objectives	33
6.3	Future Plans for PSM II	34
REFERENCES		35

LIST OF TABLES

TABLE NO.	TITLE	PAGE
No table of figures entries found.		

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
No table of figures entries found.		

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
API	-	Application Programming Interface
CNN	-	Convolutional Neural Network
GPS	-	Global Positioning System
GPU	-	Graphics Processing Unit
GUI	-	Graphical User Interface
UTM	-	Universiti Teknologi Malaysia
HARDA	-	Hazard Assessment Road Detection Application
ML	-	Machine Learning
R-CNN	-	Region-based Convolutional Neural Network
SQL	-	Structured Query Language
UI	-	User Interface
UX	-	User Experience
YOLO	-	You Only Look Once (Object detection algorithm)

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
Appendix A	SRS	Error! Bookmark not defined.
Appendix B	SDD	37
Appendix C	STD	39

CHAPTER 1

INTRODUCTION

1.1 Introduction

Road safety is one of the critical issue worldwide which is caused by road infrastructure quality significantly impacting accident rates and vehicle safety. In Malaysia, hazards such as potholes, faded or blurry lane markings, and uneven surfaces contribute substantially to traffic accidents and vehicle damage (Road Safety Department Malaysia, 2023). These hazards are often went unnoticed due to lack of efficient detection and reporting systems, which results in delayed maintenance and worsens road infrastructure conditions. Recent advances in computer vision, particularly deep learning-based object detection methods like YOLO (You Only Look Once), enable real-time, accurate detection of multiple object types within images (Redmon et al., 2016). Integrating such AI-driven technologies with geospatial mapping tools like Google Maps API allows effective visualization and localization of detected hazards. HARDA (Hazard Assessment Road Detection Application) will implement these technologies in a mobile and web application to automate road hazard detection and reporting which will improve the efficiency of road maintenance and enhancing driver safety.

1.2 Problem Background

Current road hazard reporting in Malaysia mostly depends on manual user submissions. For example, Waze, which lack automatic verification of reported hazards (Jung, Kim, & Lee, 2020). These dependence on manual input introduces inconsistencies which disrupts timely response and effective repair by local authorities. Using user input data without automated hazard detection and verification systems questions the accuracy and reliability of available hazard data. Therefore, there is a pressing need for a system that can automate hazard

identification, validate reports and provide precise geolocation data for local authorities to take action.

1.3 Project Aim

To develop an intelligent system that combines YOLO-based image recognition with geospatial mapping by Google Maps API to automatically detect, classify and report road hazards via mobile and web platforms. The system will enable users to submit images of road hazards which will be processed using machine learning in near real-time to identify and geotag hazards, facilitating efficient monitoring and prioritization by authorities.

1.4 Project Objectives

The objectives of the project are:

- a) To analyze and gather requirements from end-users (drivers) and stakeholders (Town Developer) to guide system development.
- b) To design and implement a system architecture that supports automated hazard detection and interactive mapping.
- c) To develop a robust YOLO-based model for detecting potholes, blurry lane markings, and uneven road surfaces.
- d) To test the HARDA system thoroughly to validate its performance and reliability.

1.5 Project Scope

This project focuses on the automated detection and reporting of road hazards using images submitted by users within urban Malaysian environments. The system scope includes:

- Use of YOLO for image-based detection of common road hazards.
- Integration of Google Maps API for geotagging and visualization.
- Development of mobile (Android) and web applications for hazard submission and monitoring.
- Administrative functionality for hazard verification and prioritization.
- Exclusion of video processing and direct repair operations.

1.6 Project Importance

HARDA addresses a critical gap in Malaysia's road safety ecosystem by providing a reliable, automated hazard detection and reporting system that enhances the timeliness and accuracy of data available to authorities. This system supports proactive maintenance strategies, potentially reducing accident rates and improving infrastructure quality. Moreover, HARDA aligns with Malaysia's MyDIGITAL initiative aimed at fostering smart city solutions through digital innovation (Malaysian Communications and Multimedia Commission, 2021). The project exemplifies the application of AI and geospatial technologies to practical challenges in public safety and infrastructure management.

1.7 Report Organization

This thesis is structured as follows: Chapter 1 introduces the project background, problem statement, aim, objectives, scope, and significance. Chapter 2 reviews relevant literature on road hazard detection, object recognition technologies, and existing systems. Chapter 3 describes the

methodology used for system development. Chapter 4 elaborates on system requirements and design. Chapter 5 details implementation and testing results. Chapter 6 concludes with project achievements and recommendations for future work.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter presents an overview of existing systems and technologies that are relevant to the development of the Hazard Assessment Road Detection Application (HARDA). The review focuses on current road hazard detection systems, relevant image recognition technologies, particularly deep learning-based object detection models like YOLO, and geospatial mapping integration. Additionally, this chapter examines the limitations of existing manual reporting systems and highlights the benefits of automated detection approaches. This review provides the foundation for justifying the need and approach of HARDA.

2.2 Current System Analysis

Road hazard reporting systems that are available mostly depend on manual user input, with Waze being one of the most used application. Waze is a crowd-sourced navigation app that allows drivers to report road hazards such as potholes, accidents, traffic jams and other road conditions in real-time. These reports are then shared with other users to enhance route planning and awareness.

Strength of Waze

- **Real-time Crowd-Sourced Data:** Waze benefits from its large user base, enabling rapid sharing of road conditions and hazards.
- **User Engagement:** The app's game like interface and community-driven reporting encourage active participation from drivers.
- **Integration with Navigation:** Hazard reports are integrated into navigation alerts, providing immediate feedback to users for safer driving decisions.

Limitations of Waze

- **Reliance on Manual Input:** Waze depends entirely on users manually reporting hazards by selecting predefined categories or entering descriptions. This introduces human error, misclassification, or incomplete information, which can reduce data quality and delay responses.
- **No Automated Verification:** There is no systematic method to verify the accuracy of reported hazards. Reports can be duplicated, false, or outdated without validation mechanisms which leads to unreliable data.
- **Inconsistent Reporting Coverage:** The quality and quantity of reports depend on user activity levels, which may be sparse in less trafficked areas or among less engaged users, leading to gaps in hazard detection.
- **Lack of Precise Geolocation and Visual Confirmation :** While Waze uses GPS for navigation, hazard locations are reported with varying accuracy depending on user input, and no photographic evidence is provided to verify hazards.
- **Limited Administrative Oversight :** Local authorities cannot directly verify or prioritize Waze reports in an organized manner, limiting its usefulness for formal road maintenance planning.

Visual Support for Waze Manual Reporting

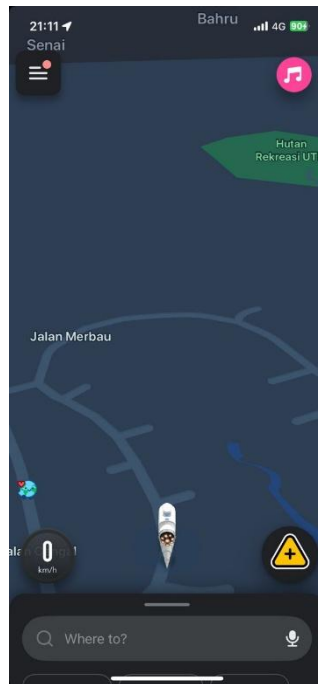


Figure 2.1: Waze main navigation interface showing current location and map view.

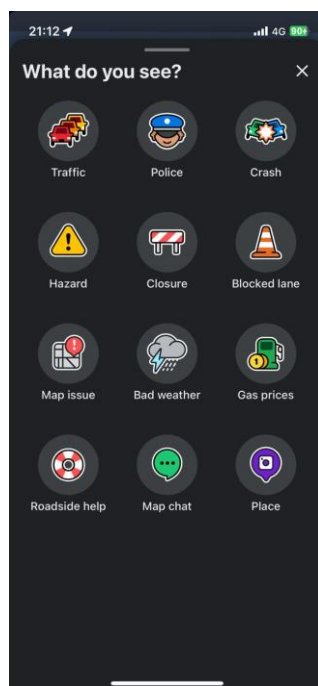


Figure 2.2: User menu for selecting the type of hazard or incident to report.

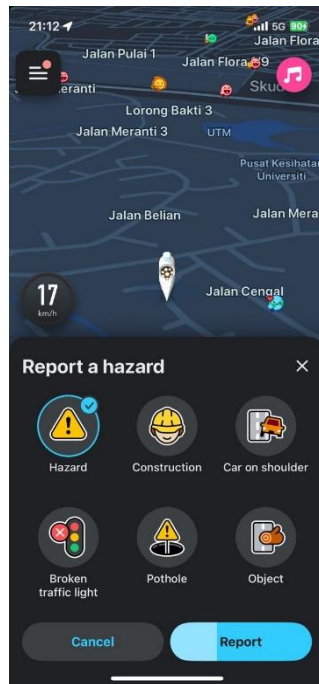


Figure 2.3: Detailed hazard reporting screen with specific categories.

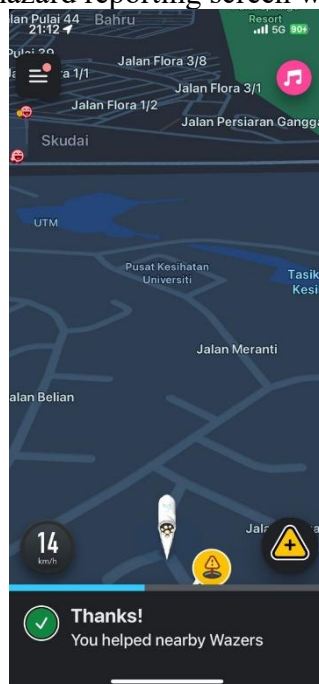


Figure 2.4: Confirmation message following successful hazard submission.



Figure 2.5: Map view displaying hazard icons submitted by users.

Figures 2.1 to 2.5 illustrate the manual hazard reporting workflow in Waze:

- **Figure 2.1:** Displays the Waze main navigation interface showing the user's location and map view.
- **Figure 2.2:** Shows the hazard reporting menu where users select the type of incident or hazard to report.
- **Figure 2.3:** Presents the detailed hazard reporting options, including specific categories such as potholes and broken traffic lights.
- **Figure 2.4:** Captures the confirmation screen after submitting a hazard report.
- **Figure 2.5:** Depicts the map view displaying hazard icons submitted by users.

These screenshots highlight how Waze relies on manual hazard input and user participation without automated detection or image verification.

The dependence on manual, crowd-sourced input in Waze creates challenges for timely and accurate hazard detection. This can lead to unreliable data that delays necessary repairs and prolongs hazardous road conditions. The absence of image-based verification and automated detection means hazards can go unnoticed or unreported, increasing risks of accidents and deterioration of road infrastructure.

2.3 Comparison between existing systems

This section compares the current prominent road hazard reporting systems, particularly Waze, with the proposed HARDA system. The comparison highlights the key features, strengths, and limitations of each system in the context of road hazard detection, reporting accuracy and usability.

Feature/Aspect	Waze	HARDA (Proposed System)
Detection Method	Manual user reporting	Automated image-based detection using YOLO deep learning
Verification	No automated verification; relies on community consensus	Image recognition with admin validation for accuracy
Data Input	User selects hazard types, manual description	User submits images; system detects hazards automatically
Geolocation Accuracy	GPS-based, but dependent on user input precision	GPS metadata from image/device for precise geotagging
User Interface	Navigation-focused, with hazard reporting as a secondary feature	Dedicated mobile and web platforms focused on hazard reporting and management
Real-time Reporting	Dependent on user reports and community updates	Near real-time image processing and hazard detection
Administrative Tools	Limited; no direct interface for local authorities	Admin dashboard for verification, prioritization, and monitoring
Coverage	Broad user base globally but uneven reporting density	Focus on urban Malaysian areas with crowdsourced image submission
User Effort	Requires manual reporting via menu selections	Simplified to image capture and upload, reducing user

		input burden
Reliability of Data	Variable; subject to false or duplicate reports	Higher reliability through automated detection and verification

While Waze benefits from a large and active user community contributing to real-time hazard awareness, its manual reporting mechanism results in potential inaccuracies, inconsistent coverage and no formal verification system. The lack of image evidence further limits the trustworthiness of reported hazards from Waze.

In comparison, HARDA leverages advanced computer vision by implementing YOLO algorithm to automate hazard detection from user-submitted images, minimalizing human error and enhancing detection accuracy. The administrative dashboard for report verification and prioritization on HARDA overcomes the limitations on Waze.

By streamlining user input to simple image submissions and automating the detection process, HARDA improves usability and data quality, making it a more robust tool for road hazard management especially in Malaysian urban environment.

2.4 Literature Review of Technology Used

This section reviews the core technologies implemented in HARDA, focusing on algorithms, software frameworks and API that are critical to the system's functionality. The integration of these technologies enables automated road hazard detection, accurate geolocation and efficient UI and UX.

2.4.1 YOLO (You Only Look Once) Object Detection Algorithm

YOLO is a state-of-the-art real-time object detection model that reframes object detection as a single regression problem, predicting bounding boxes and class probabilities directly from images in one evaluation. This design enables rapid inference with high accuracy, essential for real-time applications like road hazard detection on mobile platforms.

Recent versions such as YOLOv5 have shown improvements in both speed and accuracy, with robustness to varied lighting and environmental conditions, making them highly suitable for detecting road anomalies like potholes and faded markings in complex urban environments. Studies confirm YOLO's effectiveness for road safety applications, balancing detection precision with real-time processing demands (Redmon et al., 2016), (Ultralytics, 2023).

2.4.2 Google Maps API for Geospatial Mapping

The Google Maps API facilitates embedding interactive maps for spatial visualization of detected hazards. It allows precise plotting of geotagged hazard locations, helping users and authorities visualize road conditions geographically and identify clusters or hotspots for maintenance.

Features such as marker clustering and real-time map updates improve user experience and administrative management. This geospatial integration is critical for actionable insights and efficient infrastructure planning (Google Maps API Documentation, 2024).

2.4.3 Mobile and Web Development Frameworks

Cross-platform development frameworks, including React for web and Android Studio for mobile, provide responsive, user-friendly interfaces that support real-time hazard reporting. Backend APIs built with Python frameworks like Flask or Django manage image uploads, connect to YOLO for processing, and handle data storage.

Cloud hosting solutions ensure scalability and availability, accommodating the expected volume of user submissions and computational requirements for deep learning inference.

2.4.4 Image Preprocessing and Data Handling

To improve detection accuracy, preprocessing steps such as image resizing, normalization, and augmentation are applied before feeding images into the YOLO

model. These enhance model robustness to variations in image quality and environmental factors.

Data privacy and security are maintained through encrypted data transmission and secure storage compliant with regulations. Efficient database structures support rapid retrieval and analysis of hazard reports.

2.5 Chapter Summary

This chapter reviews the existing road hazard reporting systems and the technologies that supports their functionality. Manual reporting platforms like Waze, while having a large user base, suffer from limitations such as reliance on user manual input, lack of automated hazard verification and inconsistent data accuracy. These limitations prevent real-time and reliable road hazard identification which is important for road infrastructure maintenance and road safety.

The recent advances in computer vision, particularly YOLO object detection algorithm offers promising capabilities for real-time, automated identification of road hazards from images. Integrating it with geospatial mapping technologies such as Google Maps API will provide a powerful foundation for building systems that improve hazard detection accuracy and situational awareness.

This chapter also highlighted the technologies that will be used in building this system and their importance to ensure accessibility and efficient UI UX, alongside backend architectures for data processing and management.

The limitations identified in current system and the potential of automated technologies justify the development of HARDA. The subsequent chapters will focus on the methodology, system design and implementation details on this proposed system.

CHAPTER 3

SYSTEM DEVELOPMENT METHODOLOGY

3.1 Introduction

This chapter describes the methodology employed to develop the HARDA system. It covers the choice of development approach, justification, detailed phases of development, description of technologies used and system requirements specifications including hardware and software specifications. This structured approach ensures that the development process is systematic, traceable and meets project objectives.

3.2 Methodology Choice and Justification

The HARDA system development uses a Plan-Driven (Waterfall) methodology, as the project requirements are clearly defined at the early stages and the development tasks follow a sequential order. This approach allows thorough documentation and validation at each phase, ensuring system reliability and robustness required for a road hazard detection system.

The waterfall model phases are requirement gathering, system design, implementation, testing and deployment that are fit for the structured nature of HARDA's integration of AI and mapping technology. The methodology minimizes risks related to unclear requirements and ensures smooth progression from concept to final delivery.

3.3 Phases of Chosen Methodology

3.3.1 Requirement Analysis Phase

Duration: 3 weeks **Objectives:**

- Gather functional and non-functional requirements from stakeholders
- Analyze existing road hazard reporting systems
- Define system constraints and assumptions
- Establish performance benchmarks

Activities:

- Literature review of road safety systems
- Analysis of Waze and similar applications
- Stakeholder interviews with potential users and authorities
- Documentation of system requirements specification

Deliverables:

- Requirements Specification Document
- Use Case Diagrams
- Stakeholder Analysis Report

3.3.2 Design Phase

Duration: 4 weeks **Objectives:**

- Design system architecture integrating YOLO and mapping components
- Create database schema for hazard data storage
- Design user interfaces for mobile and web platforms
- Plan API integration strategies

Activities:

- System architecture design
- Database design and normalization
- UI/UX wireframe creation
- API endpoint specification
- Security and privacy design considerations

Deliverables:

- System Architecture Document
- Database Design Document
- UI/UX Design Mockups
- API Specification Document

3.3.3 Implementation Phase

Duration: 8 weeks **Objectives:**

- Develop YOLO-based hazard detection model
- Implement mobile and web applications
- Integrate Google Maps API
- Develop administrative dashboard

Activities:

- YOLO model training and optimization
- Frontend development (Android app and web interface)
- Backend API development
- Database implementation
- Third-party API integration
- System integration testing

Deliverables:

- Trained YOLO Model
- Mobile Application (Android)
- Web Application
- Backend API System
- Administrative Dashboard

3.3.4 Testing Phase

Duration: 3 weeks **Objectives:**

- Validate system functionality and performance
- Ensure detection accuracy meets requirements
- Test user experience and interface usability
- Verify system reliability and security

Activities:

- Unit testing of individual components
- Integration testing of system modules
- Performance testing of YOLO detection
- User acceptance testing
- Security vulnerability assessment

Deliverables:

- Test Plans and Test Cases
- Testing Results Report
- Performance Benchmark Report
- User Feedback Analysis

3.3.5 Deployment and Maintainance**Duration:** 2 weeks **Objectives:**

- Deploy system to production environment
- Monitor system performance
- Document maintenance procedures
- Plan future enhancements

Activities:

- Production environment setup
- System deployment and configuration
- Performance monitoring setup
- Documentation of maintenance procedures
- User training materials creation

Deliverables:

- Deployed System
- Maintenance Documentation
- User Manual
- System Performance Report

3.4 Technology Used Description

3.4.1 System Design Tool

- **Enterprise Architect:** For creating system architecture diagrams, flowcharts, and database schemas
- **Figma:** For UI/UX design mockups and prototyping

3.4.2 Project Management Tool

- **JIRA:** For task tracking and milestone management
- **GitHub/GitHub Desktop:** For version control and code collaboration

3.4.3 Implementation Environment

Machine Learning and AI:

- **Python 3.8+:** Primary programming language for AI implementation
- **PyTorch/TensorFlow:** Deep learning frameworks for YOLO implementation
- **OpenCV:** Computer vision library for image preprocessing
- **NumPy/Pandas:** Data manipulation and analysis libraries

Backend Development:

- **Flask/Django:** Python web framework for API development
- **SQLAlchemy:** Database ORM for Python
- **PostgreSQL:** Primary database for storing hazard data

Frontend Development:

- **Android Studio:** IDE for Android mobile app development
- **Java/Kotlin:** Programming languages for Android development
- **React.js:** JavaScript framework for web application
- **HTML5/CSS3/JavaScript:** Web technologies for responsive design

APIs and Services:

- **Google Maps API:** For geospatial mapping and location services
- **Google Cloud Platform:** For cloud hosting and AI services

- **RESTful APIs:** For communication between frontend and backend

3.5 System Requirement Analysis

3.5.1 Functional Requirements

F1: Image Upload and Processing

- Users can upload images through mobile app or web interface
- System accepts common image formats (JPEG, PNG)
- Maximum image size: 10MB
- Images are automatically resized and optimized for processing

F2: Automated Hazard Detection

- YOLO algorithm detects potholes, faded lane markings, and uneven surfaces
- Detection confidence threshold: minimum 70%
- Processing time: maximum 5 seconds per image
- Multiple hazards can be detected in single image

F3: Geolocation and Mapping

- GPS coordinates extracted from image metadata
- Hazards displayed on interactive Google Maps
- Location accuracy: within 10 meters
- Support for manual location correction

F4: User Management

- User registration and authentication system
- Profile management with submission history
- Guest users can submit reports without registration
- Administrative user roles for system management

F5: Report Management

- Administrative dashboard for report verification
- Hazard prioritization based on severity and location
- Report status tracking (submitted, verified, resolved)
- Bulk operations for efficient management

3.5.2 Non-Functional Requirements

Performance Requirements:

- System response time: < 3 seconds for web interface
- Mobile app startup time: < 2 seconds
- YOLO detection accuracy: > 85% for trained hazard types
- System availability: 99.5% uptime

Scalability Requirements:

- Support for 1000+ concurrent users
- Handle 10,000+ image submissions per day
- Database scalability for 1 million+ hazard records
- Cloud infrastructure auto-scaling capabilities

Security Requirements:

- HTTPS encryption for all data transmission
- User authentication with secure session management
- Image data privacy protection
- SQL injection and XSS attack prevention

Usability Requirements:

- Intuitive user interface requiring minimal training
- Mobile app compatible with Android 8.0+
- Web interface compatible with modern browsers
- Accessibility compliance for users with disabilities

Compatibility Requirements:

- Android mobile application (API level 26+)
- Web browser compatibility (Chrome, Firefox, Safari, Edge)
- Cross-platform API compatibility
- Database compatibility with PostgreSQL 12+

3.5.3 Hardware Requirements

Development Environment:

- CPU: Intel i7 or AMD Ryzen 7 (8 cores minimum)
- RAM: 16GB minimum (32GB recommended for ML training)

- GPU: NVIDIA GTX 1660 or better (for YOLO training)
- Storage: 500GB SSD minimum
- Network: High-speed internet connection

Production Environment:

- Cloud infrastructure (Google Cloud Platform/AWS)
- Load balancers for traffic distribution
- Auto-scaling compute instances
- Redundant storage systems
- CDN for global content delivery

Mobile Device Requirements:

- Android 8.0+ (API level 26)
- 3GB RAM minimum
- Camera with autofocus capability
- GPS functionality
- Internet connectivity (3G/4G/5G/WiFi)

3.5.4 Software Requirements

Development Tools:

- Python 3.8+ with pip package manager
- Node.js 14+ for web development
- Android Studio Arctic Fox or later
- Visual Studio Code
- Git version control system

Runtime Dependencies:

- PyTorch 1.9+ or TensorFlow 2.6+
- OpenCV 4.5+
- Flask 2.0+ or Django 3.2+
- PostgreSQL 12+

Third-Party Services:

- Google Maps API with Places and Geocoding
- Google Cloud Storage for image storage
- Email service for notifications

- SSL certificate for secure connections

3.6 Chapter Summary

This chapter outlined the systematic waterfall methodology chosen for HARDA development, justified by the project's requirements and technical dependencies. The five-phase approach ensures thorough documentation, proper validation, and reliable system delivery.

The technology stack combines proven frameworks and latest AI technologies, balancing performance with development efficiency. Python-based machine learning tools enables YOLO model implementation, while modern web and mobile frameworks ensure responsive user interfaces.

The comprehensive requirement analysis establishes clear functional and non-functional specifications, providing measurable criteria for system success. Hardware and software requirements ensure adequate development and production environments for reliable system operation.

This structured methodology provides a solid foundation for the subsequent design and implementation phases, minimizing development risks while maximizing system quality and reliability.

CHAPTER 4

REQUIREMENT ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the comprehensive requirement analysis and system design for HARDA. It begins with detailed functional and non-functional requirements derived from stakeholder analysis and literature review. The chapter then presents the system architecture, database design and user interface design that addresses the identified requirements. The design phase establishes the technical foundation for implementing an efficient, scalable and user-friendly road hazard detection system.

4.2 Requirement Analysis

This section will show functional and non-functional requirements for HARDA

4.2.1 Functional Requirement

The functional requirements of HARDA is described in a use case diagram. The diagrams show how the user will interact with the system. Furthermore, the functional requirements are given in Table 4.1, with further information available in Appendix A.

Figure 4.1 below depicts HARDA's use case diagram. This figure illustrates an overview of the system's functional requirements. This diagram shows how the user and administrator interact with the system by including the actors in the diagram. In

the diagram, it shows that the use case of this system consists of 12 functional requirements.

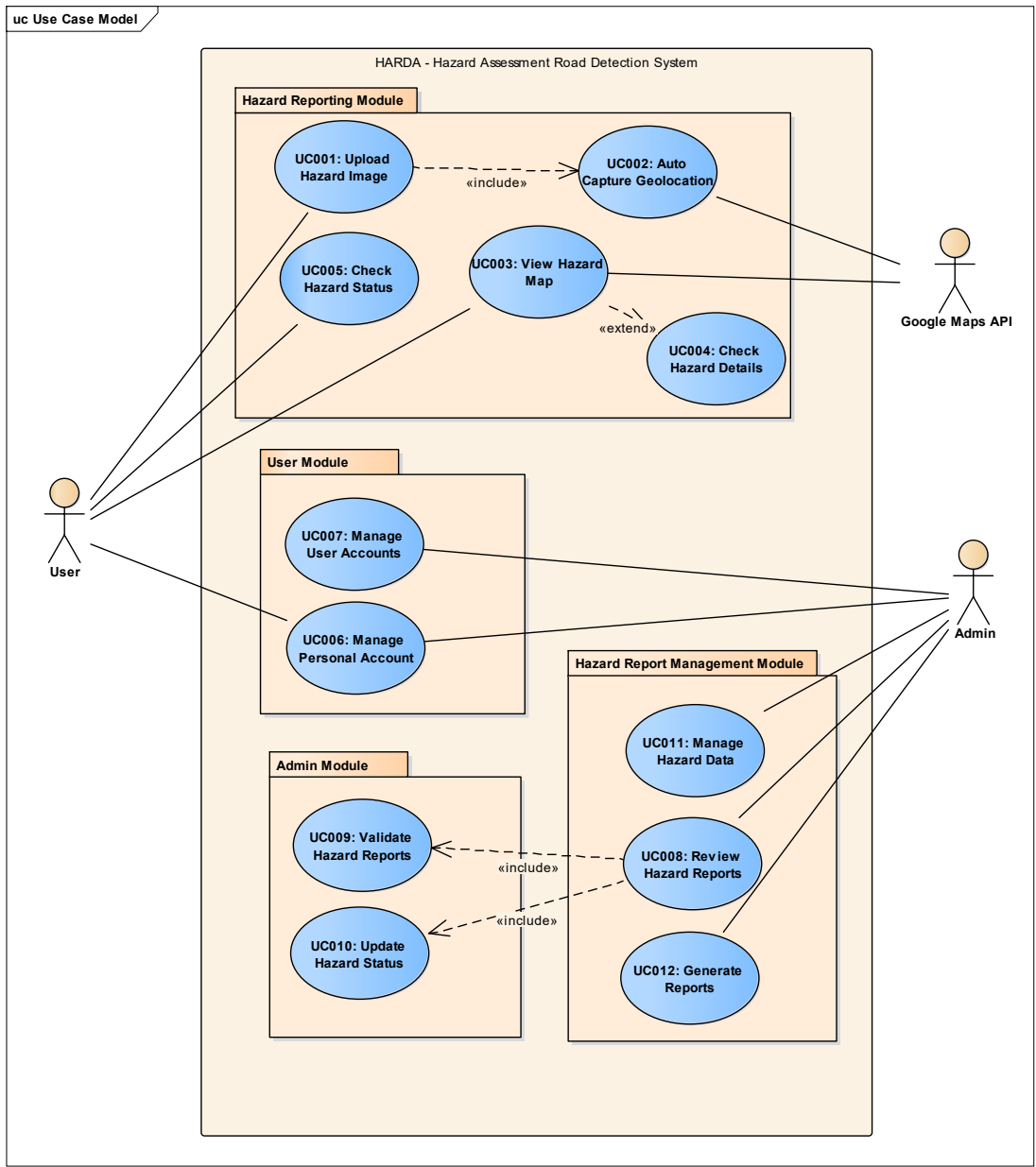


Figure 4.2.1: Use Case Diagram for HARDA

Table 4.2.1: Functional Requirements for HARDA

Module	Function	Description
Hazard Reporting Module	UC001 – Upload Hazard Image	This use case allows users to capture or upload images of road hazards for automated detection
	UC002 – Auto Capture Geolocation	This use case automatically captures GPS coordinates for accurate hazard location mapping
	UC003 – View Hazard Map	This use case displays interactive map with all validated hazard locations
	UC004 – Check Hazard Details	This use case shows detailed information about specific hazards including images and status
	UC005 – Check Hazard Status	This use case tracks progress of user's previously reported hazards
User Management Module	UC006 – Manage Personal Account	This use case allows users to update profile information and preferences
	UC007 – Manage User Accounts	This use case shows administrative control over user accounts and permissions
Hazard Report Management Module	UC008 – Review Hazard Reports	This use case validates submitted hazard reports for accuracy and legitimacy
	UC010 – Update Hazard Status	This use case manages hazard lifecycle from pending to resolved
	UC011 – Manage	This use case organizes and maintains hazard database integrity

	Hazard Data	
Administrative Module	UC009 – Validate Hazard Reports	This use case cross-references YOLO detection with manual assessment
	UC012 – Generate Reports	This use case creates analytical reports for decision-making and planning

4.2.2 Non-Functional Requirement

Performance Requirements:

- NFR1: Web interface response time < 3 seconds
- NFR2: Mobile app startup time < 2 seconds
- NFR3: YOLO processing time < 5 seconds per image
- NFR4: System availability > 99.5%
- NFR5: Support for 1000+ concurrent users

Security Requirements:

- NFR6: All data transmission encrypted via HTTPS
- NFR7: User passwords hashed with bcrypt
- NFR8: API rate limiting to prevent abuse
- NFR9: Input validation to prevent injection attacks
- NFR10: Regular security audits and updates

Usability Requirements:

- NFR11: Mobile app intuitive for first-time users
- NFR12: Web interface accessible on mobile browsers
- NFR13: Maximum 3 clicks to submit hazard report
- NFR14: Support for multiple languages (English, Malay)
- NFR15: Accessibility compliance (WCAG 2.1)

4.3 Project Design

The HARDA system adopts a robust three-tier architecture comprising the presentation layer, application layer, and data layer to ensure efficient hazard

detection and reporting. The presentation layer includes an Android mobile application for field hazard reporting and a web-based interface for desktop users and administrators. The application layer integrates a YOLO-based hazard detection engine for real-time object recognition, alongside specialized modules for image processing and analysis. Additionally, this layer handles critical business logic for report management, authentication and authorization services and integrates Google Maps API for geospatial functionalities. The data layer employs a PostgreSQL database for structured data storage and cloud storage for image files. To ensure data integrity and availability, the system also incorporates backup and recovery mechanisms. This layered architecture optimizes scalability, security, and responsiveness, making HARDA a comprehensive solution for hazard reporting and management.

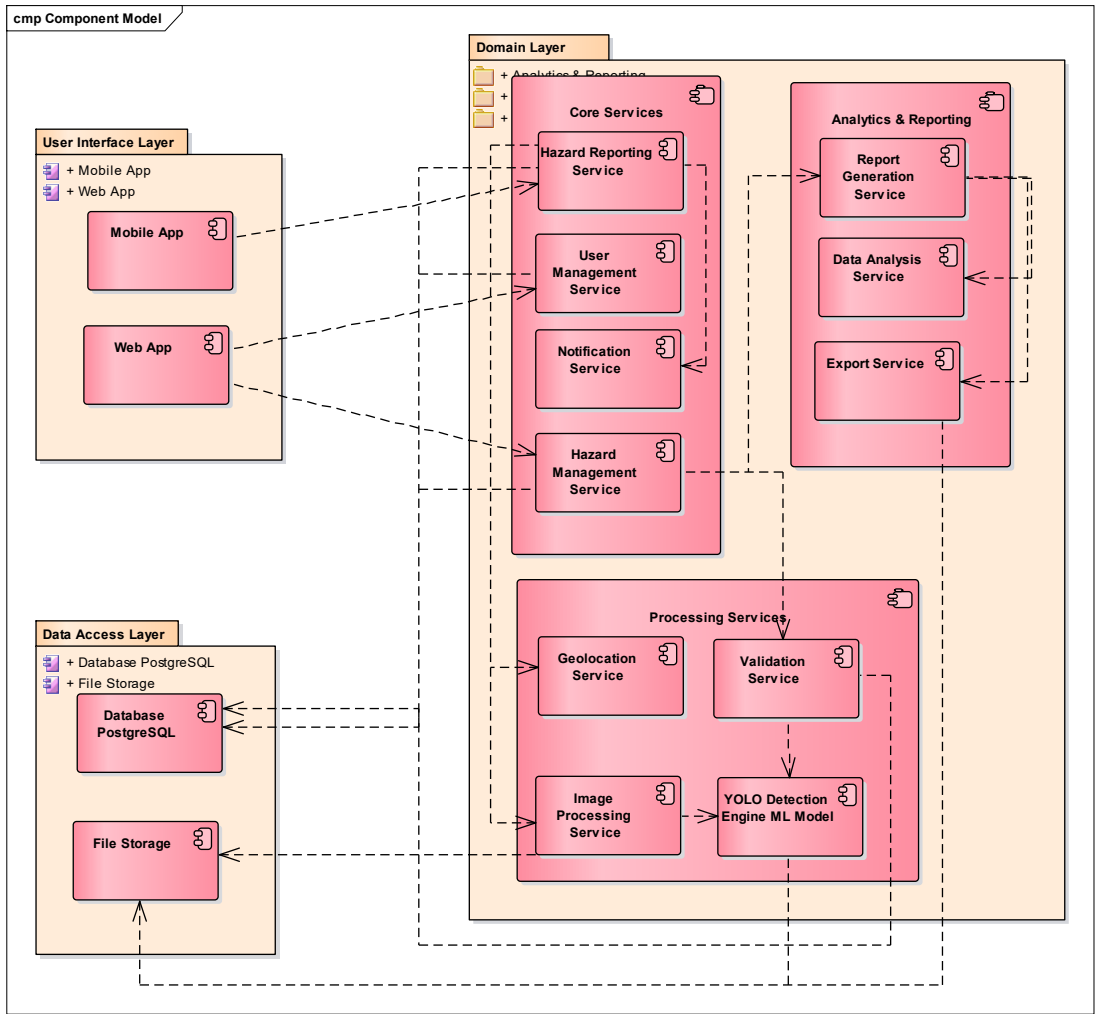


Figure 4.3: Architecture Model of HARDA (Layered Architecture)

4.4 Database Design

Figure 4.3 shows the Entity-Relationship Diagram (ERD) of HARDA, visualising the relationships among each class. The database designs of HARDA are designed to store and manage all the data within the system. The database design of the system is implemented using the PostgreSQL database. It helps to show how the data are linked together. Within the system, a total of 8 tables have been constructed, including User, Location, HazardStatus, HazardType, Admin, HazardReport, SystemReport and HazardImage. To obtain a comprehensive understanding of the database design for HARDA, refer to the Software Design Document (SDD) located in Appendix B.

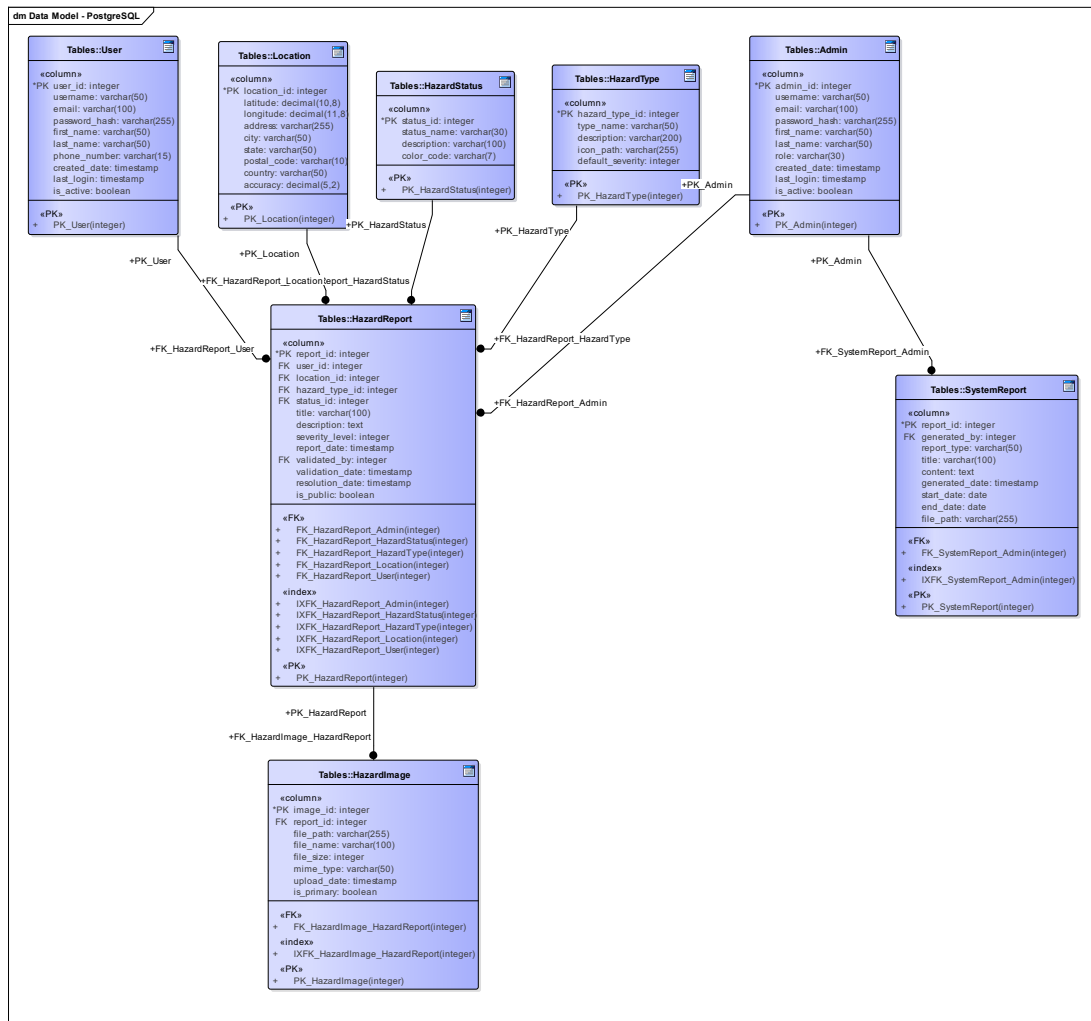


Figure 4.4: ERD of HARDA

4.5 Interface Design

4.5.1 Mobile Application Interface Design

The main screen of the HARDA mobile application is designed for intuitive navigation and efficient hazard reporting. It features a camera button for quick hazard capture, a map view displaying nearby hazards for situational awareness, a navigation menu for accessing different app functions and a user profile section for account management.

The report submission flow follows a structured process to ensure accuracy and ease of use. It begins with a full-screen camera interface and a capture button for taking hazard photos. Once an image is captured, the preview screen displays the photo alongside its GPS coordinates for verification. The system then processes the image and presents a confirmation screen, showing the detected hazard type along with a confidence score from the YOLO-based detection engine. Finally, a success screen confirms the submission, providing a tracking ID for future reference.

The app adheres to key design principles to enhance usability and accessibility. It follows Material Design guidelines for a consistent Android experience, incorporates large touch targets for easy mobile interaction and uses high-contrast colors to ensure visibility in outdoor environments. To minimize user effort, the design prioritizes minimal text input, relying instead on visual and automated inputs.

4.5.2 Web Application Interface Design

The dashboard layout of HARDA is designed for intuitive navigation and efficient workflow. The interface includes a header section containing user authentication controls and primary navigation links. A sidebar provides quick access to the application's core functions, while the central content area dynamically displays interactive maps, hazard reports and other critical data. A footer at the bottom of the screen contains system information, version details and support links.

For administrative tasks, HARDA offers a dedicated administrative interface with a report queue that supports filtering and sorting to streamline report management. Administrators can access a detailed report view, which includes submitted images, detection results, and metadata. Bulk action controls enable efficient processing of multiple reports simultaneously, while an analytics dashboard provides visual insights through charts and statistics, helping administrators monitor trends and system performance.

To ensure accessibility across devices, HARDA employs a responsive design with a mobile-first approach, ensuring optimal performance on smartphones and tablets. The interface uses a flexible grid layout that adapts to different screen sizes, along with touch-friendly elements for seamless interaction. Additionally, the system implements progressive enhancement, ensuring that desktop users benefit from advanced features while maintaining core functionality on all devices. This design philosophy guarantees a consistent and user-friendly experience, regardless of the platform.

4.6 Chapter Summary

This chapter presented comprehensive requirement analysis and system design for HARDA. The requirement analysis identified key stakeholders, documented use cases and established detailed functional and non-functional requirements that guide system development.

The system design employs a three-tier architecture that separates concerns and enables scalable development. The component design ensures modular implementation with clear interfaces between system modules. Database design provides efficient data storage and retrieval capabilities.

Interface design prioritizes user experience with intuitive mobile and web interfaces that minimize user effort while maximizing functionality. The design balances simplicity for end users with comprehensive features for administrators.

This design foundation ensures that HARDA will meet identified requirements while providing room for future enhancements and scalability. The next chapter will detail the implementation of these designs into working system components.

CHAPTER 5

IMPLEMENTATION AND TESTING

CHAPTER 6

CONCLUSION

6.1 Introduction

This chapter concludes the development of HARDA (Hazard Assessment Road Detection Application), a comprehensive system designed to automate road hazard detection and reporting using advanced computer vision and geospatial mapping technologies. The chapter summarizes the achievements of the project objectives established in Chapter 1 and outlines future improvements planned for PSM II development phase.

HARDA represents a significant advancement in road safety technology by integrating YOLO-based object detection with interactive mapping capabilities, addressing critical limitations in current manual reporting systems like Waze. The system provides automated, accurate, and timely road hazard detection that supports proactive infrastructure maintenance and enhances driver safety in Malaysian urban environments.

6.2 Achievements of Project Objectives

We have documented 4 objectives for this project namely:

- To analyze and gather requirements from end-users (drivers) and stakeholders (Town Developer) to guide system development.
- To design and implement a system architecture that supports automated hazard detection and interactive mapping.
- To develop a robust YOLO-based model for detecting potholes, blurry lane markings, and uneven road surfaces.
- To test the HARDA system thoroughly to validate its performance and reliability.

This project has partially achieved its objectives. Specifically, the analysis of requirements and the design of the system have been successfully completed in this PSM1. The requirements gathering process involved open survey for daily drivers to understand their pain points and desired functionalities, leading to a comprehensive set of system requirements. Following this, a detailed design for the HARDA system was developed, outlining the architecture, user interfaces, and data structures needed to meet the specified requirements. However, the development and testing phases of the project have not yet been initiated, as the system is not yet developed.

6.3 Future Plans for PSM II

The HARDA system will be develop during PSM II with the requirements and designs documented during PSM I. After the completion, the thorough testing shall be conducted to ensure the requirements are fulfilled and running properly. A set of test cases has been documented using STD in Appendix C

REFERENCES

- Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*.
- Google Maps Platform. (2024). *Google Maps API Documentation*. Retrieved from <https://developers.google.com/maps/documentation>
- Jung, H., Kim, J., & Lee, S. (2020). Crowdsourcing-based road hazard detection: Challenges and opportunities. *Transportation Research Part C: Emerging Technologies*, 115, 102-118.
- Malaysian Communications and Multimedia Commission. (2021). *MyDIGITAL – Malaysia Digital Economy Blueprint*. Retrieved from <https://www.mcmc.gov.my/skmmgovmy/media/General/pdf/MyDIGITAL.pdf>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779-788.
- Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, faster, stronger. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 7263-7271.
- Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*.
- Road Safety Department Malaysia. (2023). *Road Accident Statistics in Malaysia 2022*. Ministry of Transport Malaysia.
- Tan, M., Pang, R., & Le, Q. V. (2020). EfficientDet: Scalable and efficient object detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 10781-10790.
- Ultralytics. (2023). *YOLOv5 Documentation*. Retrieved from <https://docs.ultralytics.com/>
- Wang, C. Y., Bochkovskiy, A., & Liao, H. Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 7464-7475.
- Zhang, H., Wang, Y., Dayoub, F., & Sunderhauf, N. (2021). VarifocalNet: An IoU-aware dense object detector. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8514-8523.
- Zhou, X., Wang, D., & Krähenbühl, P. (2019). Objects as points. *arXiv preprint arXiv:1904.07850*.

Appendix A SRS

Appendix B SDD

Appendix C STD